m43-temas-avanzados-sql-solucion

October 18, 2026

1 Módulo 43: Temas avanzados de SQL — Solución

Author: Juliho David Castillo Colmenares

Enunciado para estudiantes: m43-temas-avanzados-sql-guia.ipynb · Lección: m43-temas-avanzados-sql-leccion.ipynb

1.1 Contexto

La actividad pide proponer un conjunto de tablas que se puedan relacionar, como se hizo en el módulo y de preferencia relacionadas con el área de trabajo, crear con ellas una base de datos relacional en Python con SQLite a partir de archivos CSV y realizar consultas relacionales entre ellas. Esta solución modela una tienda con clientes y pedidos.

1.2 Datos

- `customers.csv` (20 filas, UTF-8): `customer_id` (clave primaria), `customer_name`, `address`, `city` y `country`.
- `orders.csv` (40 filas): `order_id` (clave primaria), `customer_id` (clave foránea hacia `customers`), `order_date` y `total_purchase` (importe del pedido).

1.3 Instrucciones

1. **Crear la base de datos relacional** con las tablas propuestas, explicando su aplicación y el significado de cada campo.
2. **`INNER JOIN` y `LEFT JOIN`:** dos consultas específicas, explicando la lógica de cada una.
3. **Clasificar con `CASE WHEN`:** una consulta que use `CASE WHEN`, indicando el problema que resuelve.
4. **Tres subconsultas** (semi-join, anti-join u otra), explicando la lógica de cada una.

Entrega: un Jupyter Notebook (`.ipynb`) con el código de la base de datos y las consultas y la explicación de cada consulta; los archivos CSV utilizados, y un PDF con capturas de pantalla de la ejecución del código, los resultados de las consultas y los comentarios pertinentes a cada pregunta. Las entregas sin estos elementos no se califican. Tiempo estimado: 40 minutos.

1.3.1 Tarea 1. Crear la base de datos relacional

Se crean las tablas `Customers` y `Orders` y se cargan desde los CSV con una función genérica.

[1]:
```
import sqlite3
import csv
import time

# Connect to the SQLite database (it will be created if it doesn't exist)
now = str(int(time.time()))
conn = sqlite3.connect(f"my_database_{now}.db")
cursor = conn.cursor()


# Function to load data from a CSV file into a table
def load_data_from_csv(csv_file, table_name, columns):
    with open(csv_file, "r", encoding="utf-8") as file:
        dict_reader = csv.DictReader(file)
        to_db = [tuple(row[col] for col in columns) for row in dict_reader]

    cursor.execute(f'CREATE TABLE IF NOT EXISTS {table_name} ({", "
 ↪join(columns)});')
    cursor.executemany(
        f'INSERT INTO {table_name} ({", ".join(columns)}) VALUES ({", ".join(["?
 ↪" for _ in columns])});',
        to_db,
    )
    conn.commit()


# Define the columns and load data from Customers and Orders
customers_columns = ["customer_id", "customer_name", "address", "city",␣
 ↪"country"]
orders_columns = ["order_id", "customer_id", "order_date", "total_purchase"]

load_data_from_csv("customers.csv", "Customers", customers_columns)
load_data_from_csv("orders.csv", "Orders", orders_columns)
```

1.3.2 Tarea 2. INNER JOIN y LEFT JOIN

[2]:
```
# Query to perform INNER JOIN between Customers and Orders
inner_join_query = """
SELECT Customers.customer_name, Orders.order_date, Orders.total_purchase
FROM Customers
INNER JOIN Orders ON Customers.customer_id = Orders.customer_id;
"""
cursor.execute(inner_join_query)
inner_join_results = cursor.fetchall()

print("INNER JOIN Results:")
# for row in inner_join_results:
```

```
#     print(row)
inner_join_results[:5]
```

```
INNER JOIN Results:
```

[2]:
```
[('Juan Perez', '2023-01-15', '250'),
 ('Juan Perez', '2024-08-05', '50'),
 ('Maria Lopez', '2023-02-10', '100'),
 ('Maria Lopez', '2024-09-20', '400'),
 ('Carlos Sanchez', '2023-03-05', '75')]
```

[3]:
```
# Query to perform LEFT JOIN between Customers and Orders
left_join_query = """
SELECT Customers.customer_name, Orders.order_date, Orders.total_purchase
FROM Customers
LEFT JOIN Orders ON Customers.customer_id = Orders.customer_id;
"""
cursor.execute(left_join_query)
left_join_results = cursor.fetchall()

print("\nLEFT JOIN Results:")
# for row in left_join_results:
#     print(row)
left_join_results[:5]
```

```
LEFT JOIN Results:
```

[3]:
```
[('Juan Perez', '2023-01-15', '250'),
 ('Juan Perez', '2024-08-05', '50'),
 ('Maria Lopez', '2023-02-10', '100'),
 ('Maria Lopez', '2024-09-20', '400'),
 ('Carlos Sanchez', '2023-03-05', '75')]
```

1.3.3 Tarea 3. Clasificar con CASE WHEN

[4]:
```
# Query to categorize total purchases into Low, Medium, and High categories␣
 ↪using CASE WHEN
case_when_query = """
SELECT Orders.customer_id,
       CASE
           WHEN CAST(Orders.total_purchase AS REAL) < 100 THEN 'Low'
           WHEN CAST(Orders.total_purchase AS REAL) BETWEEN 100 AND 500 THEN␣
 ↪'Medium'
           ELSE 'High'
       END AS purchase_category
FROM Orders;
"""
```

```
cursor.execute(case_when_query)
case_when_results = cursor.fetchall()

print("\nCASE WHEN Results:")
# for row in case_when_results:
#     print(row)
case_when_results[:5]
```

```
CASE WHEN Results:
```

[4]:
```
[('1', 'Medium'),
 ('2', 'Medium'),
 ('3', 'Low'),
 ('4', 'Medium'),
 ('5', 'High')]
```

1.3.4 Tarea 4. Tres subconsultas

[5]:
```
# Query to return customers who have placed at least one order (Semi-Join)
semi_join_query = """
SELECT customer_name
FROM Customers
WHERE EXISTS (SELECT 1 FROM Orders WHERE Customers.customer_id = Orders.
 ↪customer_id);
"""
cursor.execute(semi_join_query)
semi_join_results = cursor.fetchall()

print("\nSemi-Join Results (Customers with orders):")
# for row in semi_join_results:
#     print(row)
semi_join_results[:5]
```

```
Semi-Join Results (Customers with orders):
```

[5]:
```
[('Juan Perez',),
 ('Maria Lopez',),
 ('Carlos Sanchez',),
 ('Ana Gomez',),
 ('Pedro Rodriguez',)]
```

[6]:
```
# Query to return customers who have not placed any orders (Anti-Join)
anti_join_query = """
SELECT customer_name
FROM Customers
```

```
WHERE NOT EXISTS (SELECT 1 FROM Orders WHERE Customers.customer_id = Orders.
 ↪customer_id);
"""
cursor.execute(anti_join_query)
anti_join_results = cursor.fetchall()

print("\nAnti-Join Results:")
# for row in anti_join_results:
#     print(row)
anti_join_results[:5]
```

```
Anti-Join Results:
```

[6]:
```
[]
```

Scalar subquery: orders above the average purchase. The inner query `(SELECT AVG(...) FROM Orders)` runs once and returns a single value, the average purchase over all orders. The outer query compares each order against that value and keeps only the ones above it, so the threshold adapts automatically if new orders are loaded. Since the tables were created without column types, `total_purchase` is stored as text; `CAST(... AS REAL)` makes the comparison numeric instead of alphabetical.

[7]:
```
# Query to return the orders whose amount is above the average purchase (Scalar␣
 ↪Subquery)
scalar_subquery_query = """
SELECT order_id, customer_id, order_date, total_purchase
FROM Orders
WHERE CAST(total_purchase AS REAL) > (SELECT AVG(CAST(total_purchase AS REAL))␣
 ↪FROM Orders);
"""
cursor.execute(scalar_subquery_query)
scalar_subquery_results = cursor.fetchall()

print("\nScalar Subquery Results (Orders above the average purchase):")
# for row in scalar_subquery_results:
#     print(row)
scalar_subquery_results[:5]
```

```
Scalar Subquery Results (Orders above the average purchase):
```

[7]:
```
[('104', '4', '2023-03-20', '500'),
 ('105', '5', '2023-04-11', '700'),
 ('107', '7', '2023-06-15', '300'),
 ('110', '10', '2023-09-20', '400'),
 ('111', '11', '2023-10-11', '600')]
```

1.3.5 Cierre

[8]:
```
# Close the SQLite connection
conn.close()
```

Revisión posterior (2026-10): se añadió un tercer ejemplo de subconsulta (una subconsulta escalar que compara cada orden con la compra promedio), con su explicación, porque el enunciado pide tres y había dos. Además, la consulta `CASE WHEN` convierte `total_purchase` a número con `CAST(... AS REAL)`: las columnas se crean sin tipo y se guardan como texto, así que la comparación era alfabética y todas las órdenes salían como 'High'. Los CSV se leen ahora con `encoding="utf-8"` para conservar los acentos. El PDF conserva la versión entregada.